

UNIVERSIDADE ESTADUAL DE PONTA GROSSA
SETOR DE ENGENHARIAS, CIÊNCIAS AGRÁRIAS E DE
TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA

Carlos Eduardo Varela de Almeida
Mateus Enrick Pitura
Raissa Mayara Moreira

Impacto do WebAssembly no desempenho de
aplicações Rust

PONTA GROSSA
2026

Carlos Eduardo Varela de Almeida

Mateus Enrick Pitura

Raissa Mayara Moreira

Impacto do WebAssembly no desempenho de aplicações Rust

Trabalho de conclusão de Curso
apresentado ao Curso de Engenharia
de Software da Universidade Estadual
de Ponta Grossa, como requisito parcial
para obtenção do título de Bacharel em
Engenharia de Software.

Orientador: Prof. Dr. Luciano José Senger

PONTA GROSSA
2026

AGRADECIMENTOS

Expressamos nossos sinceros agradecimentos ao Prof. Dr. Luciano José Senger pela orientação dedicada ao longo deste trabalho, conduzida com atenção, paciência e compromisso acadêmico. Suas contribuições foram essenciais para o desenvolvimento e a conclusão desta pesquisa.

Aos nossos pais e amigos, manifestamos nossa profunda gratidão pelo apoio incondicional, incentivo constante e confiança depositada durante toda a trajetória acadêmica. A presença de vocês foi fundamental para a concretização deste trabalho.

RESUMO

O WebAssembly tem se consolidado como uma alternativa para execução de aplicações de alto desempenho em navegadores *web*, permitindo a utilização de linguagens compiladas além da linguagem JavaScript. Entre elas, o Rust destaca-se pela eficiência computacional e sua segurança de memória, tornando-se uma opção importante para o desenvolvimento de aplicações executadas no ambiente *web*. Neste contexto, este trabalho realiza uma análise comparativa entre a execução nativa de algoritmos implementados em Rust e a execução dos mesmos algoritmos compilados para WebAssembly. Para a realização dos experimentos, foram selecionados algoritmos representativos de diferentes classes de problemas computacionais a partir do The Computer Language Benchmarks Game. As implementações foram adaptadas para os dois ambientes de execução e avaliadas em ambiente controlado por contêineres Docker, com coleta de métricas de tempo de execução, consumo de memória e utilização de CPU. Os resultados preliminares mostram que o *overhead* do WebAssembly em tempo de execução cresce conforme o padrão de acesso à memória de cada algoritmo, variando de 2,8% nos casos com acesso sequencial a 110% no algoritmo mais intensivo. As análises de CPU e memória indicam que parte desse impacto está relacionada ao *overhead* do ambiente, e não apenas à execução dos algoritmos, sendo necessário considerar métricas como *baseline* e consumo líquido para uma avaliação mais precisa. Espera-se que os resultados finais contribuam para a compreensão dos impactos do WebAssembly em aplicações Rust e auxiliem na tomada de decisões sobre quando utilizar essa tecnologia em aplicações executadas no navegador.

Palavras-chave: Rust; WebAssembly; desempenho; benchmarking; execução nativa; Docker; Puppeteer.

LISTA DE FIGURAS

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos .	21
Figura 2 – Comparativo de tempo de execução médio	26

LISTA DE TABELAS

Tabela 1 – Comparativo de tempo de execução médio (ms)	25
Tabela 2 – Consumo de memória do executável Rust (MiB)	27
Tabela 3 – Consumo de memória do executável WebAssembly (MiB)	27

LISTA DE ALGORITMOS

Algoritmo 1	–	Exemplo de <i>ownership</i> : cada valor tem um único dono	11
Algoritmo 2	–	Violação das regras de <i>borrowing</i> em Rust	12
Algoritmo 3	–	Referências separadas por escopo, <i>borrowing</i> correto	12
Algoritmo 4	–	Acesso fora dos limites com encerramento seguro	13
Algoritmo 5	–	Tipo <i>Option</i> no lugar de ponteiro nulo	13
Algoritmo 6	–	Função Rust exportada para WebAssembly	15
Algoritmo 7	–	Chamada da função Rust a partir do JavaScript	16
Algoritmo 8	–	Memória liberada automaticamente ao fim do escopo	17
Algoritmo 9	–	Função Rust em Wasm recebendo <i>string</i> do JavaScript	18

LISTA DE ABREVIATURAS E SIGLAS

WASM	WebAssembly
W3C	World Wide Web Consortium
HTML	HyperText Markup Language
URL	Uniform Resource Locator
HTTPS	Hypertext Transfer Protocol Secure
JIT	Just-In-Time
LLVM	Low Level Virtual Machine
CPU	Central Processing Unit
RAM	Random Access Memory
SSD	Solid State Drive
NVMe	Non-Volatile Memory Express
SATA	Serial AT Attachment
WSL	Windows Subsystem for Linux
GB	Gigabyte
MiB	Mebibyte
MHz	Megahertz

SUMÁRIO

1	INTRODUÇÃO	9
1.1	OBJETIVOS	9
1.1.1	Objetivo geral	9
1.1.2	Objetivos Específicos	9
1.2	JUSTIFICATIVA	10
2	REVISÃO DA LITERATURA	11
2.1	RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE	11
2.1.1	Segurança: o Modelo de Ownership e Borrowing	11
2.2	WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS	14
2.2.1	Arquitetura e Funcionamento	14
2.2.2	Segurança	15
2.2.3	Compilação de Rust para WebAssembly	15
2.3	ESTUDOS COMPARATIVOS DE DESEMPENHO	16
2.3.1	Rust: Desempenho sem Runtime ou Garbage Collector	17
2.3.2	WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa	18
3	MATERIAL E MÉTODOS	19
3.1	PESQUISA BIBLIOGRÁFICA	19
3.2	IMPLEMENTAÇÃO PRÁTICA	19
3.2.1	Algoritmos	20
3.2.2	Configuração dos Testes	20
3.3	AMBIENTE DE TESTE	21
3.3.1	Hardwares	21
3.3.2	Softwares	22
3.4	BENCHMARKING	23
3.4.1	Avaliação	23
4	ANÁLISES PRELIMINARES	25
4.1	TEMPO DE EXECUÇÃO	25
4.2	USO DE MEMÓRIA	26
4.3	SÍNTESE COMPARATIVA	27
5	CONSIDERAÇÕES FINAIS	29
	REFERÊNCIAS	30

1 INTRODUÇÃO

O crescimento das aplicações *web* modernas e a necessidade cada vez maior de desempenho no lado do cliente têm tornado a escolha do ambiente de execução um aspecto importante no desenvolvimento de software. Sendo assim, o WebAssembly tem se destacado por permitir a execução de código compilado diretamente no navegador com desempenho próximo ao nativo. Paralelo a isso, a linguagem Rust vem sendo utilizada como uma das principais opções para gerar código nesse formato.

Mesmo com sua popularidade, a execução de aplicações Rust por meio do WebAssembly ainda apresenta limitações, já que o ambiente do navegador impõe restrições que não existem na execução nativa. Além disso, ainda há poucos estudos experimentais que analisem de forma prática o impacto dessa abordagem sobre o desempenho.

Diante disso, surge a questão: qual é o impacto da execução de aplicações Rust via WebAssembly no desempenho quando comparado à execução nativa, considerando diferentes tipos de problemas computacionais, e em quais situações essa abordagem pode ser considerada viável?

O objetivo deste trabalho é comparar a execução nativa de programas escritos em Rust com a execução dos mesmos programas compilados para WebAssembly no navegador, analisando o impacto em desempenho e consumo de recursos. Para isso, foram utilizados algoritmos representativos do The Computer Language Benchmarks Game (Fulgham; Gouy, 2026), adaptados para execução nos dois ambientes e avaliados sob condições controladas para coleta de métricas comparáveis.

1.1 OBJETIVOS

1.1.1 Objetivo geral

Este trabalho tem como objetivo geral realizar uma análise comparativa entre a execução de código Rust e a execução do mesmo código compilado para WebAssembly no navegador, avaliando os impactos em desempenho e consumo de recursos.

1.1.2 Objetivos Específicos

- Selecionar algoritmos representativos de diferentes classes de problemas computacionais a partir do The Computer Language Benchmarks Game (Fulgham; Gouy, 2026), adequados para execução nos dois ambientes avaliados;

- Avaliar o consumo de recursos computacionais e tempo de execução de cada um dos algoritmos escolhidos;
- Elaborar diretrizes sobre em quais cenários a execução via WebAssembly representa uma alternativa viável à execução nativa.

1.2 JUSTIFICATIVA

O avanço das aplicações *web* modernas tem impulsionado a busca por tecnologias capazes de oferecer desempenho próximo ao nativo diretamente no navegador. Nesse cenário, a combinação entre Rust e WebAssembly desponta como uma alternativa promissora, reunindo as garantias de segurança e eficiência da linguagem à portabilidade do formato binário suportado pelos principais navegadores.

Embora existam estudos comparando execução nativa e WebAssembly, estes concentram-se majoritariamente em linguagens como C e C++, deixando Rust pouco explorado nesse contexto (Jangda *et al.*, 2019). Este trabalho se justifica, portanto, tanto pelo valor técnico de produzir dados comparativos concretos sobre os dois modelos de execução quanto pela contribuição ao campo da Engenharia de Software, oferecendo subsídios para decisões arquiteturais em projetos que avaliem a viabilidade de adotar WebAssembly como ambiente de execução.

2 REVISÃO DA LITERATURA

Nesta seção são abordados os principais fundamentos e características técnicas das tecnologias Rust e WebAssembly. Inicialmente, apresenta-se a linguagem Rust, sua performance e sua proposta de garantir segurança de memória sem o uso de coletor de lixo, delegando o controle de alocação ao compilador por meio do modelo de *ownership*. Em seguida, discute-se o uso de WebAssembly como padrão binário de baixo nível suportado pelos principais navegadores, abordando sua arquitetura, modelo de execução e o processo de compilação de código Rust para esse formato.

2.1 RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE

Rust é uma linguagem de programação de sistemas desenvolvida a partir da necessidade de conciliar alto desempenho e segurança no desenvolvimento de software. Desde que a Mozilla começou a patrocinar seu desenvolvimento em 2010, a linguagem foi ganhando espaço na comunidade devido à forma como ela lida com dois aspectos historicamente desafiadores no desenvolvimento de software: o desempenho e a segurança de memória.

2.1.1 Segurança: o Modelo de Ownership e Borrowing

Se o desempenho de Rust já se mostra expressivo, é na segurança que a linguagem realmente se destaca. Estima-se que cerca de 70% das vulnerabilidades de segurança catalogadas pelo Microsoft Security Response Center têm origem em erros de gerenciamento de memória (Bugden; Alahmar, 2022). Nesse contexto, o Rust aborda diretamente esse problema por meio de um mecanismo denominado *ownership* (propriedade).

O conceito é baseado em um princípio: cada valor em Rust tem um único dono, responsável por seu gerenciamento. Quando o proprietário deixa de existir ou sai de escopo, o valor associado é automaticamente desalocado da memória. Isso impede, por exemplo, que um programa tente usar um dado que já foi descartado, erro conhecido como *use after free*, comum em C e frequentemente explorado por hackers.

Algoritmo 1 – Exemplo de *ownership*: cada valor tem um único dono

```
fn main() {  
    let nome = String::from("Rust");  
    // O ownership de 'nome' é transferido para 'linguagem'.  
    let linguagem = nome;
```

```
// A linha abaixo causaria erro de compilação,  
// porque 'nome' não é mais o dono do valor.  
println!("{}", nome); // erro!  
println!("{}", linguagem); // funciona normalmente  
}
```

Fonte: Os autores

Quando há necessidade de compartilhar um valor entre diferentes partes do programa sem transferir o *ownership*, existe o *borrowing* (empréstimo). As regras são: ou várias partes do código leem o dado ao mesmo tempo, referências imutáveis, ou uma única parte pode modificá-lo, referência mutável. As duas situações nunca coexistem (Bugden; Alahmar, 2022).

Essa regra é verificada pelo compilador e impede um tipo de erro muito difícil de depurar chamado condição de corrida (*data race*), que acontece quando duas partes do programa tentam modificar o mesmo dado ao mesmo tempo.

Algoritmo 2 – Violação das regras de *borrowing* em Rust

```
fn processar_dados() {  
    let mut dados = String::from("exemplo");  
  
    // Cria uma referência mutável  
    let ref_mut = &mut dados;  
    ref_mut.push_str("modificado");  
  
    // Tenta criar referência imutável enquanto ref_mut existe  
    let ref_imut = &dados;  
  
    println!("{}", ref_mut);  
    println!("{}", ref_imut);  
}
```

Fonte: Os autores

O compilador rejeita esse código antes de executar. A solução é garantir que as referências não se sobreponham.

Algoritmo 3 – Referências separadas por escopo, *borrowing* correto

```
fn main() {  
    let mut valor = String::from("dados");  
  
    {  
        let leitura = &valor;  
        println!("Lendo: {}", leitura);  
    } // 'leitura' termina aqui
```

```
// Agora é seguro criar uma referência mutável
let modificacao = &mut valor;
modificacao.push_str(" modificados");
println!("Novo valor: {}", modificacao);
}
```

Fonte: Os autores

Outro ponto importante é que Rust não permite manipular endereços de memória diretamente fora de contextos explicitamente marcados como *unsafe*. Na prática, isso significa que todo acesso a listas e *arrays* é verificado automaticamente: se o programa tentar acessar uma posição inexistente, ele encerra de forma controlada em vez de continuar operando com dados corrompidos ou expondo informações que não deveria (Bugden; Alahmar, 2022). Esse tipo de proteção teria evitado o Heartbleed, uma das maiores vulnerabilidades da história da internet, que afetou uma parcela expressiva dos maiores sites HTTPS do mundo ao permitir que dados fossem lidos além dos limites permitidos da memória.

Algoritmo 4 – Acesso fora dos limites com encerramento seguro

```
fn main() {
    let numeros = vec![10, 20, 30];
    // O vetor tem 3 posições (índices 0, 1 e 2).
    // Tentar acessar o índice 5 não expõe memória indevida:
    // o programa encerra com uma mensagem de erro clara.
    println!("{}", numeros[5]); // panic: index out of bounds
}
```

Fonte: Os autores

Por fim, Rust também elimina o conceito de ponteiro nulo, aquele valor que representa "nada" e que em C e C++ frequentemente causa travamentos silenciosos. No lugar, Rust usa o tipo *Option*, que obriga o programador a decidir explicitamente o que fazer quando um valor pode não existir (Bugden; Alahmar, 2022).

Algoritmo 5 – Tipo *Option* no lugar de ponteiro nulo

```
fn buscar_nome(id: u32) -> Option<String> {
    if id == 1 {
        Some(String::from("Olá, Mundo"))
    } else {
        None // sem ponteiro nulo, sem crash silencioso
    }
}

fn main() {
    match buscar_nome(2) {
        Some(nome) => println!("Encontrado: {}", nome),
    }
}
```

```
        None => println!("Nenhum usuário com esse id"),  
    }  
}
```

Fonte: Os autores

O que torna tudo isso notável é que essas verificações acontecem em tempo de compilação. Erros que em outras linguagens só aparecem quando o programa já está rodando, às vezes em produção, às vezes explorados por um hacker, em Rust são detectados antes mesmo de o programa existir (Bugden; Alahmar, 2022).

2.2 WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS

Se Rust resolve o problema de escrever código seguro e eficiente, o WebAssembly resolve um problema diferente: como fazer esse código ser executado dentro de um navegador com desempenho próximo ao nativo. É justamente essa combinação que torna as duas tecnologias complementares e justifica o foco deste trabalho.

Por muito tempo, o JavaScript foi a única linguagem executada pelos navegadores. Isso funcionou para páginas simples, mas com o crescimento de aplicações mais pesadas, editores de imagem, jogos, processamento de vídeo, suas limitações ficaram evidentes. Foi para resolver esse problema que Google, Microsoft, Mozilla e Apple se uniram para criar o WebAssembly, lançado em 2017 como padrão oficial do W3C (Haas *et al.*, 2017).

WebAssembly é um formato binário compacto de baixo nível que executa com desempenho próximo ao nativo, servindo como alvo de compilação para linguagens como Rust, C e C++. Ele não foi criado para substituir o JavaScript, mas para trabalhar ao lado dele: enquanto o JavaScript cuida da interface e das interações, o Wasm assume as partes que exigem mais processamento (MDN Web Docs, 2026).

2.2.1 Arquitetura e Funcionamento

Um programa WebAssembly é organizado em módulos isolados, onde o código de um módulo não acessa dados de outro a menos que isso seja explicitamente permitido. O armazenamento de dados acontece em uma estrutura chamada memória linear, um vetor de bytes completamente separado do código executável e das estruturas internas do motor de execução. Isso garante que mesmo um programa com falha não consiga corromper o ambiente ao redor (Moric; Branstett; Petrunic, 2022).

Esse isolamento tem uma consequência prática importante: módulos de origens diferentes podem executar no mesmo processo sem interferir uns nos outros,

o que viabiliza o uso do Wasm em ambientes como servidores na nuvem e dispositivos embarcados, não apenas em navegadores (Moric; Branstett; Petrunic, 2022).

2.2.2 Segurança

A segurança faz parte da especificação do WebAssembly desde o início, e não foi adicionada depois como um recurso extra. Um módulo Wasm não possui acesso a nenhum recurso do sistema, arquivos, redes ou dispositivos, a menos que o ambiente hospedeiro conceda esse acesso de forma explícita. Além disso, o fluxo de execução é validado antes de o programa executar por meio de um mecanismo chamado integridade do fluxo de controle (*Control-Flow Integrity*), que impede que código malicioso redirecione a execução para posições não autorizadas na memória (Moric; Branstett; Petrunic, 2022).

Como os métodos do WebAssembly são tipados, o uso incorreto de tipos, como passar um valor do tipo errado para uma função, não é possível, o que reforça ainda mais a confiabilidade do ambiente de execução. Quando combinado com as garantias de segurança de memória do Rust, esse modelo cria uma camada dupla de proteção: o compilador Rust elimina os erros antes da geração do binário, e o WebAssembly garante que o binário gerado rode dentro de limites seguros (Moric; Branstett; Petrunic, 2022).

2.2.3 Compilação de Rust para WebAssembly

O processo de transformar código Rust em um módulo WebAssembly é direto e bem suportado pelo ecossistema da linguagem. O fluxo básico envolve três etapas: o código é escrito em Rust normalmente, compilado para o formato `.wasm` por meio de ferramentas como o `wasm-pack`, e então carregado e executado pelo ambiente de destino, seja um navegador, um servidor ou outro *runtime* compatível.

O `wasm-pack` é a principal ferramenta para esse processo no ecossistema Rust. Ela cuida de compilar o código, gerar os arquivos de integração com JavaScript e empacotar tudo em um formato pronto para ser publicado ou importado por uma aplicação *web*. Por baixo, o compilador Rust utiliza o *backend* LLVM para gerar o binário `.wasm`, o mesmo *backend* responsável pelas otimizações que tornam o código Rust tão eficiente em ambiente nativo (Moric; Branstett; Petrunic, 2022).

Algoritmo 6 – Função Rust exportada para WebAssembly

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn fibonacci(n: u32) -> u64 {
```



```

match n {
  0 => 0,
  1 => 1,
  _ => {
    let (mut a, mut b) = (0u64, 1u64);
    for _ in 2..n {
      let temp = a + b;
      a = b;
      b = temp;
    }
    b
  }
}

```

Fonte: Os autores

A macro `#[wasm_bindgen]` gera automaticamente o código de ligação (*bindings*) necessário para que essa função seja chamada a partir do JavaScript, como mostra o Algoritmo 7.

Algoritmo 7 – Chamada da função Rust a partir do JavaScript

```

import init, { fibonacci } from "../pkg/meu_projeto.js";

async function executar() {
  await init(); // carrega e instancia o módulo .wasm
  const resultado = fibonacci(30);
  console.log(resultado);
}

executar();

```

Fonte: Os autores

Vale destacar que nem todo código Rust compila para WebAssembly sem adaptações. Funcionalidades que dependem diretamente do sistema operacional, como acesso a arquivos ou criação de *threads*, não estão disponíveis no ambiente do navegador por padrão.

2.3 ESTUDOS COMPARATIVOS DE DESEMPENHO

Compreender o impacto do WebAssembly no desempenho de aplicações Rust exige fazer duas análises: primeiro, o que torna o Rust eficiente em ambiente nativo; segundo, o que acontece com esse desempenho quando o mesmo código é compilado para WebAssembly e executado dentro do navegador. As subseções a

seguir percorrem essas duas camadas, formando a base comparativa que orienta a investigação empírica deste trabalho.

2.3.1 Rust: Desempenho sem Runtime ou Garbage Collector

Quando um programa é executado, ele precisa reservar espaço na memória para guardar seus dados, e também precisa liberar esse espaço quando os dados não são mais necessários. Linguagens como Java e Python fazem isso automaticamente por meio de um mecanismo chamado coletor de lixo (*garbage collector*), que fica em segundo plano verificando o que pode ser descartado. Isso é conveniente, mas tem um custo: o coletor consome processamento, ocupa memória extra e pode pausar o programa em momentos imprevisíveis para fazer sua limpeza (Bugden; Alahmar, 2022).

Rust não tem coletor de lixo. Em vez disso, o compilador analisa o código antes de o programa ser executado e já determina exatamente quando cada dado deve ser criado e quando deve ser descartado. Quando uma variável chega ao fim do bloco em que foi criada, sua memória é liberada automaticamente, sem nenhuma instrução extra do programador e sem nenhum custo em tempo de execução.

Algoritmo 8 – Memória liberada automaticamente ao fim do escopo

```
fn main() {  
    // 'mensagem' é criada aqui e pertence a este escopo  
    let mensagem = String::from("olá, Mundo!");  
    println!("{}", mensagem);  
    // Ao chegar aqui, 'mensagem' sai de escopo  
    // e sua memória é liberada automaticamente pelo compilador.  
    // Nenhuma instrução manual é necessária.  
}
```

Fonte: Os autores

Esse modelo elimina tanto o trabalho manual de liberar memória, que em C frequentemente gera erros, quanto a sobrecarga do coletor de lixo. O resultado é um programa que roda com eficiência comparável à de C e C++, mas sem exigir que o programador gerencie a memória na mão (Bugden; Alahmar, 2022).

É esse nível de eficiência nativa que serve de referência para a comparação com o WebAssembly. Quando o mesmo código Rust é compilado para o formato .wasm e executado no navegador, parte dessas garantias de desempenho precisa ser reexaminada, pois o ambiente de execução impõe restrições e custos que não existem na compilação nativa.

2.3.2 WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa

Dentro do ambiente do navegador, o WebAssembly já representa um avanço considerável em relação ao JavaScript. Em tarefas computacionais intensas, o WebAssembly supera o JavaScript, sendo em média 1,3 vezes mais rápido que o JavaScript tradicional, seu antecessor mais próximo (Jangda *et al.*, 2019). No entanto, há uma ressalva importante: transferir dados complexos para dentro da memória do módulo tem um custo. Para passar uma *string* ao Wasm, por exemplo, é necessário codificá-la, localizar espaço livre na memória linear e escrevê-la *byte a byte*, um processo que pode anular o ganho de desempenho em operações simples com texto. Quando o dado já está na memória ou o algoritmo é suficientemente complexo, o Wasm volta a superar o JavaScript por ampla margem (Moric; Branstett; Petrunic, 2022).

Algoritmo 9 – Função Rust em Wasm recebendo *string* do JavaScript

```
use wasm_bindgen :: prelude :: * ;

#[wasm_bindgen]
pub fn contar_vogais(texto: &str) -> u32 {
    texto
        .chars()
        .filter(|c| "aeiouAEIOU".contains(*c))
        .count() as u32
}
```

Fonte: Os autores

Contudo, a comparação mais relevante para este trabalho não é entre WebAssembly e JavaScript, mas entre WebAssembly e a execução nativa do próprio Rust. Nesse confronto, o cenário é diferente. Estudos mostram que aplicações compiladas para WebAssembly são executadas, em média, entre 45% e 55% mais devagar do que o equivalente nativo, a depender do navegador utilizado (Jangda *et al.*, 2019). Esse é exatamente o *gap* que este trabalho busca caracterizar no contexto específico de aplicações Rust, contribuindo com dados concretos para orientar decisões sobre quando a portabilidade do Wasm justifica a perda de desempenho em relação à execução nativa.

3 MATERIAL E MÉTODOS

A abordagem deste trabalho é quantitativa, buscando mensurar o desempenho e o consumo de recursos de diferentes algoritmos. Para isso, são adotadas as etapas de pesquisa bibliográfica, implementação prática e análise comparativa.

3.1 PESQUISA BIBLIOGRÁFICA

Foram reunidas informações sobre Rust e WebAssembly na pesquisa bibliográfica, destacando seu contexto histórico, características técnicas e aplicabilidade.

Em relação ao Rust, foram abordados tópicos como o sistema de propriedade (*ownership*) e o modelo de empréstimo de valores (*borrowing*). Já para o WebAssembly, destacaram-se aspectos como o gerenciamento de memória, a segurança proporcionada pela restrição de permissões e sua integração com Rust por meio da portabilidade de código utilizando ferramentas como o *wasm-pack*.

3.2 IMPLEMENTAÇÃO PRÁTICA

Na etapa de implementação, foram utilizados cinco algoritmos distintos em Rust, cada um explora diferentes características de execução, como operações de inteiros e de pontos flutuantes, manipulação de *arrays*, laços de repetição, manipulação de *bits*, geração de número pseudo aleatórios e saída de dados. Os algoritmos foram obtidos do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026) e sofreram adaptações mínimas para serem executadas no ambiente de teste. Buscou-se utilizar o mínimo possível de bibliotecas externas, limitando-se à biblioteca Rayon para processamento paralelo e às bibliotecas nativas da linguagem, a fim de reduzir interferências externas nos resultados de desempenho.

No contexto do Rust, foi desenvolvido um código de entrada responsável por receber, por meio de parâmetros posicionais, o algoritmo a ser executado e a quantidade de execuções. Esse código também realiza a coleta do tempo de início e término da execução, mantendo os algoritmos isolados do processo de metrificação.

Para o WebAssembly, os algoritmos implementados em Rust foram portados utilizando a ferramenta *wasm-pack*. Além disso, foi criado um arquivo de entrada em HTML para carregar uma página contendo código JavaScript capaz de executar os algoritmos portados. Essa página também aceita parâmetros via URL para definir o algoritmo e a quantidade de execuções. A execução ocorre em um navegador sem interface gráfica (*headless*), utilizando a biblioteca Puppeteer de JavaScript.

3.2.1 Algoritmos

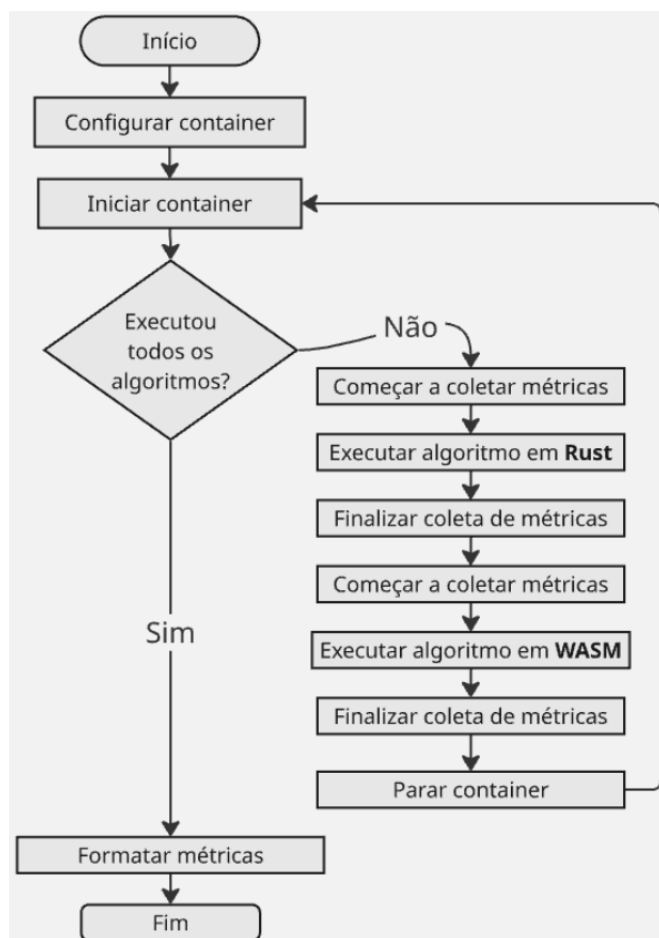
Os seguintes algoritmos foram utilizados na implementação prática:

- *fannkuch-redux*: foca em operações com inteiros e manipulação de *arrays*;
- *n-body*: realiza operações de ponto flutuante;
- *spectral-norm*: concentra-se em laços de repetição;
- *mandelbrot*: faz a manipulação de *bits*;
- *fasta*: avalia a capacidade de geração de números pseudo aleatórios e saída de dados.

3.2.2 Configuração dos Testes

Cada algoritmo foi executado 10 vezes em sequência. Foram realizados cenários com e sem 3 execuções prévias antes da carga principal, com o objetivo de avaliar o impacto da compilação JIT e do uso de *cache* para melhorar a performance. A parametrização dos algoritmos foi a mesma usada nos testes de desempenho do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026). O fluxo de execução dos testes e coleta de métricas dos algoritmos é ilustrado na Figura 1.

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos



Fonte: Os autores

3.3 AMBIENTE DE TESTE

Com o objetivo de isolar o ambiente de execução, foram utilizados contêineres por meio do Docker, com quantidades controladas de CPUs e memória RAM, visando aumentar a reprodutibilidade dos experimentos.

Os testes foram realizados em três computadores diferentes, sendo posteriormente calculada a média dos resultados obtidos. Além disso, foram padronizadas as versões das tecnologias utilizadas em todos os computadores. A seguir, são apresentados os hardwares e os softwares utilizados.

3.3.1 Hardwares

Computador 1:

- Sistema operacional: Ubuntu 24.04 (Noble);
- Kernel: Linux 6.17.0-35-generic;

- Processador: Intel i7 1165G7, 4 núcleos / 8 threads;
- Memória RAM: 16 GB LPDDR4 (8x2 GB) 4267 Mhz;
- Armazenamento: SSD NVMe de 1 TB Micron MTFDHBA1T0QFD.

Computador 2:

- Sistema operacional: Windows 11 Home Single Language 64 bits;
- Kernel: Windows NT 10.0;
- Processador: Intel Core i5-10300H, 4 núcleos / 8 threads;
- Memória RAM: 8 GB DDR (8x1 GB) 3200 Mhz;
- Armazenamento: SSD SATA de 500 GB KINGSTON SA400S37.

Computador 3:

- Sistema operacional: Ubuntu 22.04 (Jammy) via WSL;
- Kernel: Linux 6.6.114.1-microsoft-standard-WSL2;
- Processador: AMD Ryzen 5 5600x, 6 núcleos / 12 threads;
- Memória RAM: 32 GB DDR4 (2x16 GB) 3200 Mhz;
- Armazenamento: SSD NVMe de 500 GB KINGSTON SNVS500G.

3.3.2 Softwares

- Docker: versão 29.5.3;
- Docker Compose: versão v5.1.4;
- Rust: versão 1.95.0;
- wasm-pack: 0.14.0;
- wasm-bindgen: 0.2.121;
- Rayon: 1.8;
- Puppeteer: 24.43.1;
- Node.js: v24;
- npm: 10.9.4.

3.4 BENCHMARKING

Inicialmente, foi considerada a utilização de bibliotecas como Criterion e Hyperfine para a realização do *benchmarking*. Entretanto, para simplificar a implementação, foram desenvolvidos *scripts* responsáveis por automatizar a coleta de métricas utilizando o Docker, por meio do comando `docker stats`, além de código em Python para formatação e processamento dos dados coletados.

Essa abordagem também permitiu alterar parâmetros dos testes por meio de arquivos de configuração e realizar ajustes finos, buscando reduzir ao máximo interferências externas nos resultados.

3.4.1 Avaliação

Durante a avaliação, foram coletadas as métricas de tempo de execução, uso de memória ao longo do tempo e uso de CPU ao longo do tempo.

Foi utilizado um intervalo de 2 segundos entre as coletas de memória e CPU. Como essas métricas representam o consumo de recursos do contêiner, foram considerados fatores como o uso de recursos em estado de espera (*standby*) pelo contêiner e, no caso do Wasm, pelo Puppeteer, além da sobrecarga de carregamento da página *web*. Para reduzir essa interferência, foi adotado um tempo de 10 segundos em estado ocioso (*idle*) antes e depois do início dos testes. Além disso, o contêiner era parado e então iniciado antes de cada algoritmo para remover memória em *cache*.

Para a análise de desempenho temporal, o overhead percentual do WebAssembly em relação ao Rust nativo foi calculado pela equação:

$$overhead(\%) = (twasm/trust - 1) * 100 \quad (3.1)$$

Onde *trust* e *twasm* representam os tempos médios de execução em milissegundos de cada ambiente.

Para a análise de memória, foram derivadas três métricas a partir das amostras coletadas. O consumo de *baseline* corresponde ao mínimo entre as amostras em que a utilização de CPU era inferior a 10%, representando o consumo do contêiner em repouso. O consumo de pico corresponde ao máximo entre as amostras com CPU acima de 80%, representando o consumo durante a execução ativa. O consumo líquido é dado pela diferença entre pico e *baseline*:

$$baseline = \min\{mi \mid cpui < 10\%\} \quad (3.2)$$

$$pico = \max\{mi \mid cpui > 80\%\} \quad (3.3)$$

$$\text{líquido} = \text{pico} - \text{baseline} \quad (3.4)$$

Essa distinção é necessária porque o contêiner Wasm mantém o Chrome Headless e o Puppeteer em execução permanente, elevando o *baseline* a aproximadamente 200 MiB independentemente do algoritmo executado.

4 ANALISES PRELIMINARES

As métricas coletadas serão analisadas com base no levantamento teórico realizado, buscando avaliar os aspectos de desempenho de maneira objetiva e fundamentada em cada cenário analisado.

Para o tempo de execução, será calculada a média aritmética das dez iterações de cada algoritmo e comparado o resultado entre os dois ambientes, expressando a diferença como *overhead* percentual e fator multiplicativo.

Para o consumo de memória, serão aplicadas as métricas de *baseline*, pico e consumo líquido descritas anteriormente, a fim de isolar o consumo efetivo do algoritmo do *overhead* fixo do ambiente de execução Wasm.

4.1 TEMPO DE EXECUÇÃO

Cada algoritmo foi executado dez vezes em sequência, e o tempo reportado corresponde à média aritmética das execuções, em milissegundos, conforme descrito na seção anterior. A Tabela 1 apresenta os resultados obtidos, incluindo o *overhead* percentual e o fator multiplicativo calculados pelas equações apresentadas.

Tabela 1 – Comparativo de tempo de execução médio (ms)

Algoritmo	Rust (ms)	Wasm (ms)	Overhead (%)	Fator
fannkuch-redux	19.267,6	19.812,5	2,8	1,03x
spectral-norm	1.040,2	1.118,6	7,5	1,08x
fasta	1.502,5	1.791,8	19,3	1,19x
n-body	1.521,1	2.530,3	66,4	1,66x
mandelbrot	2.904,0	6.098,0	110,0	2,10x

Fonte: Os autores

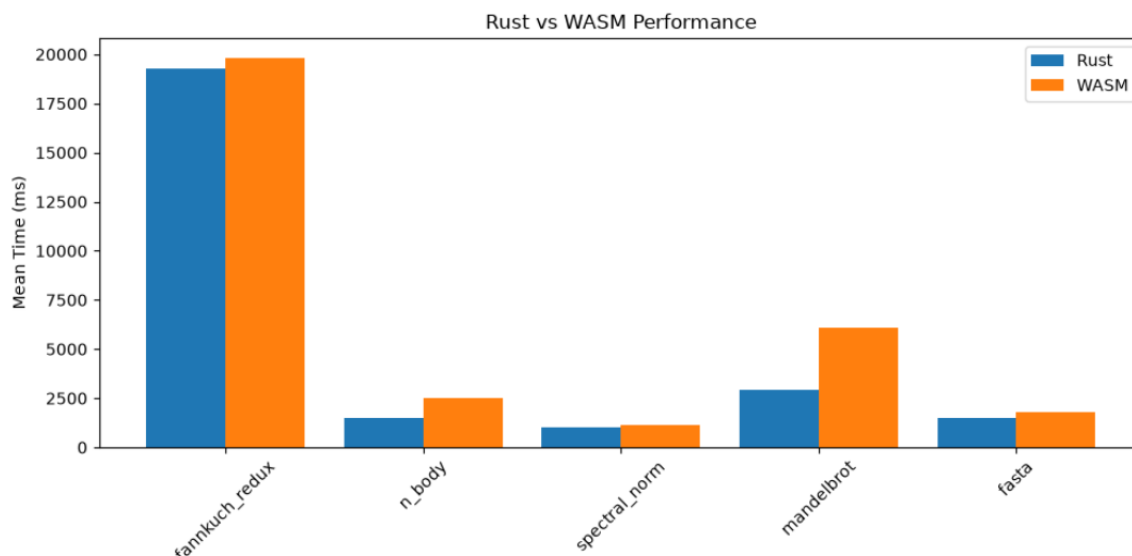
A Figura 2 apresenta a comparação dos tempos médios de execução obtidos para cada algoritmo nos ambientes Rust e WebAssembly.

O algoritmo fannkuch-redux registrou o menor *overhead*, correspondente a 2,8%, ou fator 1,03x. Trata-se de um algoritmo baseado em permutações, que utiliza operações inteiras e acesso sequencial a vetores de tamanho fixo, padrão que favorece a otimização realizada pelo mecanismo de execução do navegador.

O spectral-norm apresentou *overhead* de 7,5%. O algoritmo realiza operações de ponto flutuante com acesso predominantemente sequencial à memória, característica que também favorece sua execução em WebAssembly.

O algoritmo fasta obteve *overhead* de 19,3%. Sua execução envolve geração de sequências, consultas a tabelas de referência e escrita sequencial de dados, resultando em um padrão de acesso à memória moderadamente mais complexo.

Figura 2 – Comparativo de tempo de execução médio



Fonte: Os autores

O n-body registrou *overhead* de 66,4%. A simulação envolve cálculos iterativos de forças gravitacionais, com uso intensivo de operações de ponto flutuante de dupla precisão e acesso frequente a diferentes estruturas de dados.

O mandelbrot apresentou o maior *overhead* observado, correspondente a 110,0%. Nesse cenário, o tempo de execução em WebAssembly foi superior ao dobro do observado na execução nativa. O algoritmo realiza processamento intensivo de ponto flutuante e escrita contínua em um *buffer* bidimensional, características que aumentam a sensibilidade ao custo adicional introduzido pelo ambiente de execução.

Considerando todos os algoritmos avaliados, o *overhead* médio foi de 41,2%, enquanto a mediana foi de 19,3%. A diferença entre essas métricas indica que os casos extremos, especialmente o mandelbrot, exercem influência significativa sobre a média geral dos resultados.

4.2 USO DE MEMÓRIA

O uso de memória foi coletado por meio do comando `docker stats`, com amostragem realizada a cada dois segundos. As métricas de *baseline*, pico e consumo líquido foram calculadas conforme descrito nas seções anteriores. As Tabelas 3 e 2 apresentam os resultados obtidos.

O *baseline* do ambiente Rust situou-se entre 9,7 MiB e 12,3 MiB, refletindo principalmente o consumo do processo em estado ocioso. No ambiente WebAssembly, os valores de *baseline* variaram entre 196 MiB e 202 MiB, em razão da execução permanente do Chrome Headless e do Puppeteer.

Dessa forma, a comparação direta entre os valores totais de memória não é

Tabela 2 – Consumo de memória do executável Rust (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	12,2	52,8	40,6
n-body	12,3	55,6	43,3
spectral-norm	9,7	152,3	142,6
mandelbrot	12,2	55,5	43,2
fasta	12,2	78,7	66,5

Fonte: Os autores

Tabela 3 – Consumo de memória do executável WebAssembly (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	197,6	208,7	11,1
n-body	196,3	203,5	7,2
spectral-norm	201,1	202,3	1,2
mandelbrot	201,0	234,6	33,6
fasta	201,6	203,3	1,7

Fonte: Os autores

adequada para avaliar exclusivamente o impacto dos algoritmos. Por esse motivo, a análise foi concentrada no consumo líquido de memória.

Considerando essa métrica, o Rust apresentou valores entre 40,6 MiB, observados no fannkuch-redux, e 142,6 MiB, observados no spectral-norm. Já o WebAssembly apresentou valores entre 1,2 MiB, observados no spectral-norm, e 33,6 MiB, observados no mandelbrot.

O spectral-norm representa uma exceção no ambiente nativo, com consumo líquido de 142,6 MiB, mais de três vezes o dos demais algoritmos. Esse valor não decorre do uso da biblioteca Rayon, já que o fannkuch-redux e o mandelbrot também a utilizam e mantiveram consumo em torno de 40 MiB. O comportamento provavelmente resulta da combinação entre o curto tempo de execução do algoritmo, cerca de 1 segundo por iteração, e o intervalo de amostragem de 2 segundos, que reduz o número de amostras coletadas durante a execução ativa e torna a estimativa de pico mais sensível a variações pontuais. A verificação dessa hipótese, por meio da redução do intervalo de coleta ou do aumento da carga, está prevista para a versão final.

Esse comportamento pode ser explicado pela pré-alocação da memória linear do WebAssembly durante o carregamento da página. Como essa alocação já está contabilizada no *baseline*, ela não se reflete diretamente no consumo líquido calculado durante a execução dos algoritmos.

4.3 SÍNTESE COMPARATIVA

Os resultados obtidos permitem identificar três padrões principais.

Sensibilidade ao padrão de acesso à memória: o overhead do WebAssembly acompanha a complexidade do acesso à memória de cada algoritmo. Casos com acesso sequencial e previsível, como *fannkuch-redux* e *spectral-norm*, ficaram abaixo de 8%, próximos da execução nativa. Casos com acesso irregular ou escrita intensiva, como *n-body* e *mandelbrot*, chegaram a 66% e 110%.

Custo fixo do ambiente: o ambiente WebAssembly exige cerca de 200 MiB adicionais para manter o Chrome Headless e o Puppeteer em execução, valor praticamente constante entre os algoritmos e independente da carga processada.

Origem do custo conforme o algoritmo: para algoritmos sequenciais, o principal custo do WebAssembly está no ambiente de execução, não na execução do algoritmo em si. Para algoritmos com acesso irregular ou escrita intensiva, há também impacto relevante no tempo de execução.

5 CONSIDERAÇÕES FINAIS

O desenvolvimento deste trabalho até o presente momento permitiu consolidar fundamentos teóricos e metodológicos necessários para o andamento da análise comparativa entre a execução nativa de código rust e sua execução compilada para WebAssembly, avaliando os impactos em tempo de execução, consumo de memória e uso de CPU.

Para embasar essa análise, foram abordados os fundamentos da linguagem Rust e as características do WebAssembly na revisão da literatura. Além disso, na etapa prática foram selecionados, do The Computer Language Benchmarks Game, 5 algoritmos de diferentes classes de problemas, implementados em Rust, portados para WebAssembly e executados em ambiente controlado via Docker em três máquinas distintas. Os resultados preliminares já apontam diferenças de desempenho consistentes entre os dois ambientes, com o WebAssembly apresentando tempos de execução superiores ao Rust nativo na maioria dos algoritmos avaliados. Já as análises de uso de CPU e memória evidenciam que parte desse custo não está diretamente associada à execução dos algoritmos, mas sim ao *overhead* do ambiente de execução, especialmente no caso do WebAssembly, que mantém componentes como o Chrome Headless e o Puppeteer em funcionamento contínuo.

Diante disso, a utilização das métricas de baseline e consumo líquido mostrou-se fundamental para isolar o impacto real dos algoritmos, permitindo uma análise mais precisa e justa entre os ambientes. Observou-se ainda que o comportamento do consumo de recursos varia conforme o padrão de acesso à memória, indicando que algoritmos mais simples e com acesso sequencial tendem a apresentar desempenho mais próximo do nativo, enquanto aqueles com acesso irregular ou intensivo apresentam maior degradação de desempenho.

Como próximos passos, será concluída a análise dos dados coletados e serão apresentadas as conclusões finais do trabalho, identificando em quais cenários o WebAssembly representa uma alternativa viável à execução nativa de aplicações Rust.

REFERÊNCIAS

Bugden, W.; Alahmar, A. Rust: The programming language for safety and performance. **arXiv**, v. 2206.05503, 2022. Disponível em: <<https://arxiv.org/abs/2206.05503>>. Citado 5 vezes nas páginas 11, 12, 13, 14 e 17.

Fulgham, B.; Gouy, I. **The Computer Language Benchmarks Game**. 2026. Acesso em: 18 Jun. 2026. Disponível em: <<https://benchmarksgame-team.pages.debian.net/benchmarksgame/>>. Citado 3 vezes nas páginas 9, 19 e 20.

Haas, A. *et al.* Bringing the web up to speed with webassembly. **Communications of the ACM**, v. 61, n. 1, p. 107–115, 2017. Disponível em: <<https://dl.acm.org/doi/abs/10.1145/3062341.3062363>>. Citado na página 14.

Jangda, A. *et al.* Not so fast: Analyzing the performance of webassembly vs. native code. **Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)**, p. 107–120, 2019. Disponível em: <<https://www.usenix.org/conference/atc19/presentation/jangda>>. Citado 2 vezes nas páginas 10 e 18.

MDN Web Docs. **WebAssembly**. 2026. Acesso em: 18 jun. 2026. Disponível em: <<https://developer.mozilla.org/en-US/docs/WebAssembly>>. Citado na página 14.

Moric, Z.; Branstett, L.; Petrunic, R. Rust and webassembly for fast, secure, and reliable software. **Proceedings of the 33rd DAAAM International Symposium on Intelligent Manufacturing and Automation**, DAAAM International, Vienna, Austria, v. 33, p. 165–171, 2022. Disponível em: <<https://doi.org/10.2507/33rd.daaam.proceedings.023>>. Citado 3 vezes nas páginas 14, 15 e 18.